



Knowledge Based Systems

Reasoning with Constraints

GVHD: TS. Nguyễn Hùng Sơn

HVTH: Nguyễn Ngọc Đình – 2441862007

Hà Anh Dũng – 2441862005

Aug2025

Agenda

- Giới thiệu
- Cơ sở lý thuyết
- Phương pháp giải
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Agenda

- **Giới thiệu**
- Cơ sở lý thuyết
- Phương pháp giải
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Reasoning with Constraints

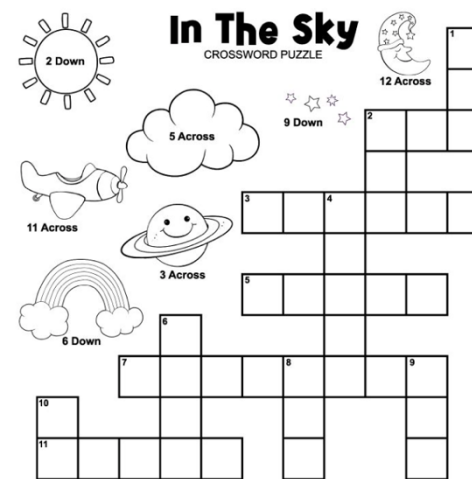
- **Reasoning with Constraints** là một lĩnh vực trong trí tuệ nhân tạo và hệ thống tri thức, nghiên cứu cách **biểu diễn tri thức dưới dạng ràng buộc** (constraints) và **tìm lời giải thỏa mãn các ràng buộc đó**.

Tại sao cần?

- Giải quyết các bài toán lập lịch, tối ưu, bài toán logic.
- Giảm không gian tìm kiếm → giải nhanh hơn brute-force.

Ví dụ ứng dụng

- Crossword Puzzle, 8-Queens Puzzle, Sudoku
- Lập lịch, phân công nguồn lực
- Tối ưu lộ trình vận tải, chuỗi cung ứng
- Thiết kế hệ thống, lập kế hoạch trong AI



Crossword puzzle

Fill in the missing numbers using 1-4 to solve the Sudoku puzzle

4	3		2
1	2		
	1		3

Sudoku

Mục tiêu và phạm vi nghiên cứu

Mục tiêu

- Hiểu **khái niệm** và **nguyên lý hoạt động** của Constraint Reasoning.
- Vận dụng vào các bài toán thực tế trong một số lĩnh vực.

Phạm vi

- Trình bày tổng quan **lý thuyết và phân loại**.
- Demo các bài toán thực tế dùng Python-Constraint và OptaPlanner.
- So sánh hiệu quả với một số phương pháp giải khác.



Agenda

- ✓ Giới thiệu
- **Cơ sở lý thuyết**
 - Nguyên lý & phân loại
- Phương pháp giải
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Nguyên lý & phân loại

Nguyên lý của bài toán

Mô tả trạng thái bằng các “**đặc trưng**” thay vì liệt kê mọi trạng thái có thể xảy ra (vốn rất nhiều)

- **Ví dụ:** Trong bài toán lập thời khóa biểu, thay vì xét toàn bộ lịch học, hãy mô tả bằng các biến như **môn học, giáo viên, phòng học, thời gian**.

Các đặc trưng đó được gắn với các biến, và giữa chúng tồn tại các ràng buộc.

- **Ví dụ:** Một giáo viên **không thể dạy 2 lớp cùng lúc** → đây là một ràng buộc cứng.

Đôi khi, có những mong muốn ưu tiên (không bắt buộc) như “**giáo viên A thích dạy vào buổi sáng**” → đây là ràng buộc mềm.

Nhiệm vụ của Reasoning with Constraints

- Tạo ra lời giải **thỏa các ràng buộc cứng** (bắt buộc)
- **Tối ưu các ràng buộc mềm** (để có lời giải “tốt nhất”)

CSP (Constraint Satisfaction Problem)

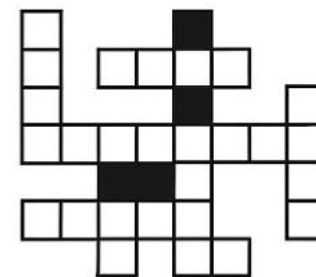
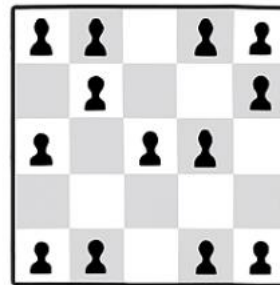
- Biểu diễn: $(X, D, C) \rightarrow \text{Solution}$
 - X: Tập biến.
 - D: Miền giá trị.
 - C: Tập ràng buộc.

Solution: Gán giá trị cho các biến thỏa mãn toàn bộ ràng buộc.

Nguyên lý & phân loại

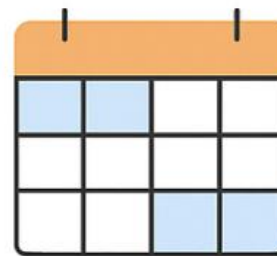
Theo loại mục tiêu

- **Satisfaction:** tìm nghiệm hợp lệ (vd: Crossword puzzle, 8-queens, Sudoku, ...)
- **Optimization:** Tìm nghiệm tối ưu (School timetabling, Vehicle routing, Call center realtime scheduling, ...)



Theo loại ràng buộc

-  **Hard constraints** (bắt buộc thoả mãn)
-  **Soft constraints** (thoả mãn nhiều nhất có thể)



Agenda

- ✓ Giới thiệu
- **Cơ sở lý thuyết**
 - ✓ Nguyên lý & phân loại
 - **Biến và phép gán**
- Phương pháp giải
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Biến và phép gán

Mỗi **đặc trưng** (feature) của một bài toán — ví dụ như “màu sắc”, “vị trí”, “ngày trong tuần”

→ được đại diện bằng **biến** (variable).

- Ví dụ: Biến X có thể đại diện cho "ngày làm việc", và **miền giá trị** của X là {Thứ Hai, Thứ Ba, ..., Thứ Sáu}
→ ký hiệu: $\text{dom}(X) = \{\text{Thứ Hai, Thứ Ba, ..., Thứ Sáu}\}$.

Biến rời rạc (discrete variable) có miền giá trị là hữu hạn hoặc vô hạn đếm được.

- Ví dụ: {đỏ, xanh, vàng} → là rời rạc, hữu hạn.

Biến nhị phân (binary variable) là biến rời rạc có đúng hai giá trị trong miền của nó.

- Ví dụ: {0, 1} hoặc {true, false} (→ gọi là biến Boolean).

Biến liên tục (continuous variable) có miền giá trị là tập số thực hoặc một đoạn trong tập số thực — ví dụ như “nhiệt độ” hoặc “thời gian”.

Biến và phép gán

Ví dụ 1: Mô hình hóa một "trạng thái" trong một lớp học

Ghi lại giờ bắt đầu → biến  **Class_time**, với các giờ hợp lệ là từ 8 giờ sáng đến 5 giờ chiều.

Ghi lại giáo viên giảng dạy → biến  **Teacher**, với tên các giáo viên hợp lệ của nhà trường.

Ghi lại môn học → biến  **Subject**, với tên các môn học hợp lệ của trường.

"**Class_time = 11, Teacher = John, Subject = Math**" → gán một giá trị cụ thể cho mỗi biến để mô tả trạng thái tại thời điểm đó.

Biến		Giá trị
Class_time		10
Teacher		John
Subject		Math

Biến và phép gán

Ví dụ 2: Trò chơi ô chữ (Crossword puzzle)

Trò chơi ô chữ (crossword puzzle) — là việc điền các từ vào lưới các ô sao cho các từ phù hợp với gợi ý.

Có hai cách khác nhau để biểu diễn trò chơi ô chữ dưới dạng các biến:

Cách 1: Biến là các từ (word)

- Các biến là **các từ** cần điền theo hướng ngang hoặc dọc.
 - Miền giá trị là tập các từ có thể được dùng cho vị trí đó.
 - Ví dụ:
 - Biến **1_across** có thể có miền giá trị là: {"ant", "big", "bus", "car", "has"}
 - Phép gán đầy đủ: gán một từ cho mỗi biến, sao cho tất cả từ điền vào đều phù hợp với gợi ý và các chữ cái giao nhau trùng khớp.
- Thích hợp khi danh sách từ có sẵn.

1	2	3
4		
5		

Row0	
Row1	
Row2	

Crossword
puzzle

Biến và phép gán

Ví dụ 2: Trò chơi ô chữ (Crossword puzzle)

Trò chơi ô chữ (crossword puzzle) — là việc điền các từ vào lưới các ô sao cho các từ phù hợp với gợi ý.



Cách 2: Biến là các ô (Cell)

- Các biến là từng ô riêng lẻ trên bảng ô chữ.
 - Miền giá trị của mỗi biến là các ký tự trong bảng chữ cái.
 - Ví dụ:
 - Ô ở góc trên bên trái là biến **p00**, với miền giá trị là {a, b, c, ..., z}.
 - Phép gán đầy đủ: gán một chữ cái cho mỗi ô vuông, và đảm bảo các ký tự tạo thành các từ hợp lệ (theo chiều ngang và dọc).
- Chi tiết hơn nhưng tăng độ phức tạp (nhiều biến hơn).

1	2	3
4		
5		

p00	p01	p02
p10	p11	p12
p20	p21	p22

Crossword
puzzle

Ví dụ 3: Phép gán đầy đủ (Total Assignments)

Giả sử có **2 biến**:

- $A \in \{0, 1, 2\}$: Biến A với miền giá trị $\{0, 1, 2\}$
- $B \in \{\text{true}, \text{false}\}$: Biến B với miền giá trị $\{\text{true}, \text{false}\}$

→ Tổng cộng có **6 phép gán đầy đủ**, được đặt tên là w_0 đến w_5 như sau:

- $w_0 = \{A = 0, B = \text{true}\}$
- $w_1 = \{A = 0, B = \text{false}\}$
- $w_2 = \{A = 1, B = \text{true}\}$
- $w_3 = \{A = 1, B = \text{false}\}$
- $w_4 = \{A = 2, B = \text{true}\}$
- $w_5 = \{A = 2, B = \text{false}\}$

Công thức tổng quát:
Tổng số phép gán đầy đủ = d^n (d: số giá trị mỗi biến, n: số biến)

Nếu có n biến, mỗi biến có d giá trị trong miền, thì tổng số phép gán đầy đủ là d^n

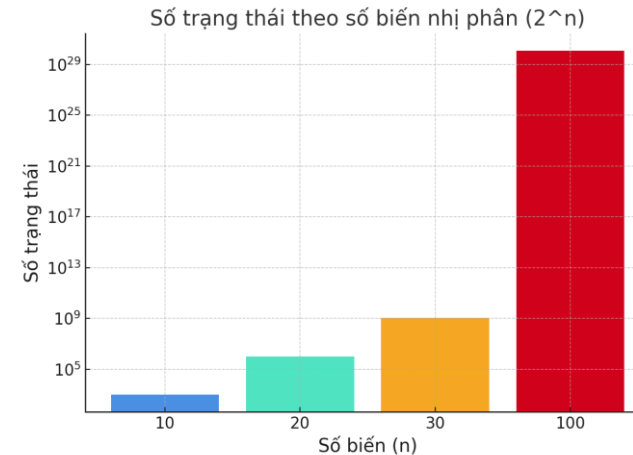
Biến và phép gán

💡 Ưu điểm của việc lập luận theo biến

Việc mô hình hoá theo **biến** giúp **tiết kiệm tính toán**.

Ví dụ:

- **10 biến** nhị phân (chỉ có 0 hoặc 1) $\rightarrow 2^{10} = \mathbf{1024}$ trạng thái.
- **20 biến** nhị phân $\rightarrow 2^{20} = \mathbf{1,048,576}$ trạng thái.
- **30 biến** nhị phân $\rightarrow 2^{30} = \mathbf{1,073,741,824}$ trạng thái.
- **100 biến** nhị phân $\rightarrow 2^{100} = \mathbf{1,267,650,600,228,229,401,496,703,205,376}$ trạng thái.



Suy luận dựa trên 30 biến có thể dễ hơn suy luận dựa trên hơn một tỷ trạng thái. Một trăm biến thì không nhiều, nhưng suy luận dựa trên hơn 2^{100} trạng thái một cách rõ ràng là không thể. Nhiều bài toán thực tế có hàng nghìn, nếu không muốn nói là hàng triệu biến.

Công thức: Số trạng thái = 2^n (n: số biến nhị phân)

Agenda

- ✓ Giới thiệu
- **Cơ sở lý thuyết**
 - ✓ Nguyên lý & phân loại
 - ✓ Biến & phép gán
 - **Ràng buộc**
- Phương pháp giải
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Ràng buộc là gì?

- Ràng buộc giống như luật chơi trong bài toán: "**Không được gán giá trị một cách tùy tiện!**".
- Ví dụ:
 - Một học sinh **không thể học hai môn cùng lúc** → ràng buộc thời khóa biểu.
 - Tổng **số người trong phòng ≤ 30** → ràng buộc an toàn.

Ràng buộc cứng (hard constraints)

- Quy định **các tổ hợp giá trị hợp lệ** giữa các biến
- Phạm vi: là các biến liên quan đến ràng buộc
- Mỗi ràng buộc là **một điều kiện logic**, đánh giá **đúng/sai** trên từng phép gán

Ràng buộc mềm (soft constraints)

- **Không bắt buộc** nhưng nên thỏa mãn nếu có thể
- Ví dụ: Giáo viên **thích dạy buổi sáng**

Các loại ràng buộc

- **Ràng buộc đơn biến (Unary constraint):** chỉ áp dụng cho **một biến duy nhất**.
 - Ví dụ: $B \leq 3$
- **Ràng buộc nhị phân (Binary constraint):** áp dụng cho **hai biến**.
 - Ví dụ: $A \leq B$
- **Ràng buộc k biến (k-ary constraint):** áp dụng cho **k biến**.
 - Ví dụ: $A + B = C$ là **ràng buộc ba biến (ternary constraint)**

Đánh giá ràng buộc trên một phép gán

- Một ràng buộc có thể được **kiểm tra (đánh giá)** trên một phép gán có **nhiều biến hơn** phạm vi của ràng buộc
→ **Các biến thừa sẽ bị bỏ qua**.
 - Ví dụ: ràng buộc $A \leq B$ vẫn đúng trong phép gán $\{A=3, B=7, C=5\}$ (biến C không liên quan).

Agenda

- ✓ Giới thiệu
- **Cơ sở lý thuyết**
 - ✓ Nguyên lý & phân loại
 - ✓ Biến & phép gán
 - ✓ Ràng buộc
 - **CSP**
- Phương pháp giải
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Constraint Satisfaction Problems

Constraint Satisfaction Problem (CSP) là gì?

CSP là **khuôn mẫu bài toán** phổ biến trong AI và khoa học máy tính. Nó giống như một **bài toán điền vào chỗ trống**, nhưng có quy tắc:

Có một số **ô trống (biến)** cần điền giá trị, mỗi ô có **tập giá trị cho phép (miền)**, và phải điền sao cho **tất cả các quy tắc (ràng buộc)** đều được thỏa mãn.

Ví dụ đơn giản

Ta cần xếp lịch cho 3 lớp học (A, B, C) vào các buổi sáng {Thứ 2, Thứ 3, Thứ 4} sao cho:

- Không có 2 lớp nào học cùng ngày
- A phải học trước B

Đây là một **bài toán CSP**, vì:

- Biến: A, B, C
- Miền: {Thứ 2, Thứ 3, Thứ 4} cho mỗi biến
- Ràng buộc: $A \neq B$, $A \neq C$, $B \neq C$, $A < B$

 Lời giải: {A = Thứ 2, B = Thứ 3, C = Thứ 4}

Constraint Satisfaction Problems

Ví dụ: Lập lịch cho robot giao hàng

Giả sử một robot giao hàng cần thực hiện 5 hoạt động: a, b, c, d, và e. Mỗi hoạt động có thể diễn ra vào thời điểm 1, 2, 3, hoặc 4.

Gọi các biến:

- A: thời điểm thực hiện hoạt động a
- B: thời điểm hoạt động b
- C: thời điểm hoạt động c
- D: thời điểm hoạt động d
- E: thời điểm hoạt động e



Miền giá trị cho mỗi biến

$\text{dom}(A)=\text{dom}(B)=\text{dom}(C)=\text{dom}(D)=\text{dom}(E)=\{1,2,3,4\}$



Các ràng buộc cần thỏa mãn

1. $B \neq 3 \rightarrow$ b không được diễn ra lúc 3
2. $C \neq 2 \rightarrow$ c không được diễn ra lúc 2
3. $A \neq B \rightarrow$ a và b không cùng thời gian
4. $B \neq C \rightarrow$ b và c không cùng thời gian
5. $C < D \rightarrow$ c phải diễn ra trước d
6. $A = D \rightarrow$ a và d diễn ra cùng lúc
7. $E < A \rightarrow$ e trước a
8. $E < B \rightarrow$ e trước b
9. $E < C \rightarrow$ e trước c
10. $E < D \rightarrow$ e trước d
11. $B \neq D \rightarrow$ b và d không cùng thời gian

Constraint Satisfaction Problems

Ví dụ 4: Lập lịch cho robot giao hàng

Bài toán thỏa mãn ràng buộc (CSP) giúp trả lời các câu hỏi quan trọng:

- Bài toán có lời giải không?
- Nếu có, tìm một lời giải hoặc tất cả lời giải.
- Tìm lời giải tốt nhất, nếu có một tiêu chí đánh giá mức độ “tốt”.
- Kiểm tra một mệnh đề có đúng với tất cả lời giải hay không.

Các bài toán CSP **xuất hiện rất phổ biến**, vì vậy việc tìm ra các **phương pháp giải hiệu quả** là điều rất đáng làm.

? Vì sao CSP khó?

- Mỗi biến là một chiều trong không gian tổ hợp
→ Có thể có hàng triệu tổ hợp gán giá trị khác nhau
- Việc kiểm tra lời giải của CSP là NP-đầy đủ (NP-complete)
→ Không có thuật toán nào giải mọi bài CSP trong thời gian đa thức.

 Tuy nhiên, **NP-đầy đủ không có nghĩa là mọi trường hợp đều khó**.

→ Rất nhiều trường hợp cụ thể có **cấu trúc đặc biệt**, và ta có thể **khai thác cấu trúc này để giải nhanh hơn**.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- **Phương pháp giải**
 - Search-based
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Solving CSPs by Searching

Giải bài toán CSP bằng Search-based

Trong CSP (Constraint Satisfaction Problem – bài toán thỏa mãn ràng buộc), ta cần tìm giá trị cho một tập hợp biến sao cho **thỏa mãn các ràng buộc** đã cho (ví dụ: sudoku, tô màu bản đồ...).

Khi bài toán có số lượng biến và miền giá trị hữu hạn, ta có thể giải nó bằng cách **thử tất cả các cách gán giá trị có thể** – gọi là "tìm kiếm toàn bộ" (brute-force).

- Duyệt tất cả các phép gán có thể.
- Kiểm tra từng phép gán xem có thỏa ràng buộc hay không.

Thuật toán generate-and-test (sinh và kiểm tra)

- Tìm một nghiệm:

- Tạo ra từng cách gán giá trị cho tất cả các biến, lần lượt kiểm tra từng cách xem nó có thỏa mãn toàn bộ ràng buộc không. Nếu có, ta dừng lại và trả về nghiệm đó.

- Để tìm tất cả các nghiệm:

- Cũng thực hiện như trên, nhưng không dừng lại ở nghiệm đầu tiên. Thay vào đó, ta tiếp tục kiểm tra cho đến hết và lưu lại tất cả các nghiệm đúng.

👉 Đây là cách giải **cơ bản và đơn giản nhất**, nhưng khi số lượng biến và giá trị lớn thì sẽ trở nên **rất tốn thời gian**, vì số lượng phép gán tăng theo cấp số mũ.

Solving CSPs by Searching

Ví dụ: Lập lịch cho robot giao hàng

Trong **Ví dụ Robot giao hàng**, không gian các phép gán (assignment space) là:

$S = \{$
 $\{A = 1, B = 1, C = 1, D = 1, E = 1\},$
 $\{A = 1, B = 1, C = 1, D = 1, E = 2\},$
 ...,
 $\{A = 4, B = 4, C = 4, D = 4, E = 4\}$

Tổng số phép gán là $|S| = 4^5 = 1024$.

Nếu có 15 biến thay vì 5 biến, thì số lượng phép gán sẽ là 4^{15} , tức khoảng **một tỷ** phép gán cần kiểm tra.

→ Không khả thi để kiểm tra toàn bộ.

Độ phức tạp khi kiểm tra ràng buộc

- Với n biến, mỗi biến có d giá trị, ta có tổng cộng d^n phép gán.
 - Nếu có e ràng buộc, tổng số lần kiểm tra là $O(e \cdot d^n)$
- Không khả thi khi n lớn.

Ý tưởng loại bỏ sớm (Pruning)

- Ràng buộc thường chỉ liên quan đến một vài biến.
 - Ta có thể kiểm tra sớm ngay khi mới gán vài biến đầu tiên.
 - Nếu một phép gán chưa hoàn chỉnh đã vi phạm ràng buộc, thì mọi phép gán đầy đủ mở rộng từ đó đều sai.
- Có thể loại bỏ sớm (prune) toàn bộ nhánh tìm kiếm.

Solving CSPs by Searching

Ví dụ: Lập lịch cho robot giao hàng



Trong bài toán **Robot giao hàng** có ràng buộc $A \neq B$.

→ Nếu ta gán $A = 1$ và $B = 1$, thì ràng buộc bị vi phạm

→ không cần xét tiếp các biến còn lại như C, D, E.



Ý tưởng:

→ Nếu phát hiện vi phạm sớm, ta có thể **loại bỏ nhánh sai** trong không gian tìm kiếm → **tiết kiệm thời gian đáng kể**.



Tìm kiếm theo chiều sâu (DFS) để giải CSP

Thuật toán DFS được dùng để tìm tất cả các nghiệm hợp lệ, được định nghĩa bởi:

- Vs: Tập biến chưa được gán trong bối cảnh hiện tại.
- Cs: Tập ràng buộc liên quan đến ít nhất một biến trong Vs.

Cách hoạt động của thuật toán

1. **Tìm tất cả ràng buộc** có thể đánh giá được trong bối cảnh hiện tại.
2. Nếu có **ràng buộc bị vi phạm**, dừng mở rộng nhánh hiện tại.
3. Nếu **tất cả biến đã được gán** và **mọi ràng buộc đều thỏa mãn** → Ghi nhận một nghiệm.
4. Nếu vẫn còn biến:
 - Chọn một biến chưa gán.
 - Thử tất cả các giá trị có thể cho biến đó.
 - Với mỗi giá trị, **gọi đệ quy** để mở rộng tìm kiếm.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- **Phương pháp giải**
 - ✓ Search-based
 - **Consistency Algorithms**
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Consistency Algorithms

Ví dụ: Lập lịch cho robot giao hàng

Mặc dù thuật toán **tìm kiếm theo chiều sâu (DFS)** giúp kiểm tra ràng buộc sớm và nhanh hơn so với phương pháp **sinh và kiểm tra (generate-and-test)**, nhưng:

👉 Vẫn còn **tốn công** nếu bài toán có **nhiều ràng buộc phức tạp** hoặc **dữ liệu lớn**.

! Hạn chế trước đây:

- **Generate-and-test**: Kiểm tra toàn bộ → rất chậm.
- **DFS + kiểm tra ràng buộc sớm**: Nhanh hơn → nhưng vẫn chưa tối ưu khi không gian tìm kiếm quá lớn.

✅ Thuật toán kiểm tra tính nhất quán (Consistency Algorithms)

Mục tiêu:

- **Giảm số lượng phép gán phải thử.**
- **Loại bỏ sớm** các giá trị không hợp lệ **ngay trong miền giá trị, trước cả khi gán.**
- **Cắt tỉa mạnh hơn** (prune sâu hơn) không gian tìm kiếm.

🧠 Ý tưởng chính:

Thay vì chỉ kiểm tra ràng buộc sau khi đã gán, ta dùng thuật toán **kiểm tra tính nhất quán** để loại bỏ trước những giá trị chắc chắn dẫn đến vi phạm.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- **Phương pháp giải**
 - ✓ Search-based
 - ✓ Consistency Algorithms
 - **Domain Splitting**
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Tách miền

Ý tưởng chính:

- Tách bài toán thành nhiều **trường hợp con không giao** nhau bằng cách chia nhỏ miền giá trị của một biến.
- Mỗi trường hợp con sẽ được giải **riêng biệt**, sau đó kết hợp nghiệm từ tất cả các trường hợp con.

Cách thực hiện:

1. Chọn một biến X .
2. **Tách miền giá trị** của X thành các trường hợp riêng biệt.
3. **Giải từng trường hợp con** bằng cách tìm kiếm hoặc kiểm tra tính nhất quán.

Ví dụ minh họa:

Giả sử biến nhị phân $X \in \{t, f\}$:

Để tìm tất cả nghiệm:

- Thử $X = t$, giải bài toán con.
- Sau đó thử $X = f$, giải bài toán còn lại.

Nếu chỉ cần một nghiệm, có thể thử $X = t$ trước. Nếu không có nghiệm, mới thử $X = f$.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- **Phương pháp giải**
 - ✓ Search-based
 - ✓ Consistency Algorithms
 - ✓ Domain Splitting
 - Local Search
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Tìm cục bộ

Các thuật toán ở các phần trước thực hiện **tìm kiếm có hệ thống** trong không gian các phép gán giá trị cho biến.

Nếu không gian là **hữu hạn**, chúng sẽ hoặc:

- **Tìm được một nghiệm**, hoặc
- **Xác nhận không có nghiệm**.

Tuy nhiên, trong nhiều bài toán thực tế:

- Không gian tìm kiếm **quá lớn** hoặc **vô hạn**.
- Việc tìm kiếm toàn bộ là **bất khả thi** trong thời gian hợp lý.

Các **phương pháp không tìm kiếm toàn bộ**, mà được thiết kế để:

- **Tìm nghiệm nhanh (trung bình)**.
- **Không đảm bảo** sẽ tìm thấy nghiệm nếu tồn tại.
- **Không thể khẳng định** rằng không có nghiệm.

Dù vậy, chúng thường được **ưu tiên** trong các ứng dụng mà:

- **Chắc chắn có nghiệm**, hoặc
- **Rất có khả năng có nghiệm và cần tìm nhanh**.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- **Phương pháp giải**
 - ✓ Search-based
 - ✓ Consistency Algorithms
 - ✓ Domain Splitting
 - ✓ Local Search
 - **COP**
- Thực nghiệm
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Tối ưu hoá

Không chỉ cần nghiệm **thỏa mãn ràng buộc**, mà còn muốn **tìm nghiệm tốt nhất** theo một tiêu chí (preference).

Bài toán tối ưu hóa (optimization problem) gồm

- **Tập biến**: mỗi biến có miền giá trị.
- **Hàm mục tiêu (objective function)**: ánh xạ mỗi nghiệm thành một số thực để đánh giá mức "tốt".

Mục tiêu

Tìm nghiệm tối ưu sao cho giá trị hàm mục tiêu nhỏ nhất (minimization) hoặc lớn nhất (maximization).

Hàm mục tiêu thường được gọi là:

- **Hàm chi phí** (cost function).
- **Hàm tổn thất / sai số** (loss function / error function).

Constraint Optimization Problems

Bài toán Constraint Optimization Problems (COP)

- Là bài toán **tối ưu hóa** trên tập nghiệm **thỏa mãn ràng buộc cứng** (hard constraints).
- Tập các nghiệm thỏa ràng buộc cứng được gọi là **tập nghiệm khả thi** (feasible assignments).
- Ngoài ra, có thể chứa **ràng buộc mềm** (soft constraints), **được chuyển thành điểm thưởng/điểm phạt** và gộp vào hàm mục tiêu.

Mục tiêu

→ Tìm **nghiệm khả thi tốt nhất** (minimize hoặc maximize) theo **giá trị hàm mục tiêu**, đồng thời **giảm vi phạm ràng buộc mềm**.

Phương pháp tiếp cận:

- Inference-based: Dùng **constraint propagation** để thu hẹp miền giá trị.
- Metaheuristic / Hybrid: Kết hợp **giải thuật di truyền, Tabu Search...** để tìm nghiệm gần tối ưu cho bài toán lớn.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Phương pháp giải
- **Thực nghiệm**
- Kết luận & Hướng phát triển
- Nguồn tham khảo



Mục tiêu thực nghiệm

Mục đích

- Kiểm chứng khả năng áp dụng các phương pháp giải CSP/COP đã trình bày vào nhiều bài toán khác nhau.
- So sánh hiệu quả giữa các công cụ (Python-Constraint, OptaPlanner) và Brute-force.
- Đánh giá sự khác biệt về hiệu năng khi số biến và độ phức tạp bài toán thay đổi.

Kết quả mong đợi

- Chứng minh các phương pháp hoạt động đúng trên nhiều dạng bài toán.
- Xác định ưu, nhược điểm của từng phương pháp và công cụ trong thực tế.

Cách tiếp cận

1. Chọn tập bài toán thử nghiệm đa dạng về loại ràng buộc và mục tiêu tối ưu:
 - Crossword Puzzle (Satisfaction)
 - N-Queens (Satisfaction)
 - School Timetabling (Mixed constraints)
 - Call Center Scheduling (Mixed constraints)
 - Vehicle Routing (Optimization)
2. Mô hình hóa mỗi bài toán dưới dạng biến – miền giá trị – ràng buộc.
3. Giải bằng các phương pháp đã trình bày, ghi nhận thời gian chạy và số nghiệm tìm được.

Test Problem 1: Crossword Puzzles

Consider the crossword puzzle shown in below Figure

1	2	3
4		
5		

Words:

add, age, aid, aim, air,
are, arm, art, bad, bat,
bee, boa, dim, ear, eel,
eft, lee, oaf

A crossword puzzle to be solved with six words

Variables

- Mỗi từ (word) cần điền vào ô chữ (slot) tương ứng.

Hard constraints

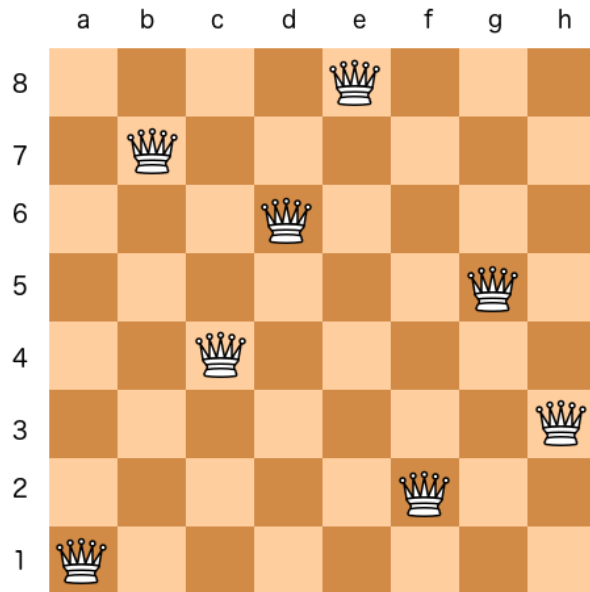
- Tất cả các từ trong ô chữ phải khác nhau.
- Các ký tự ở các ô giao nhau giữa dòng và cột phải khớp nhau.

Soft constraints

- Không có.

Test Problem 2: N-Queens

Hãy xét bài toán N-Queens với $N = 8$:



Đặt N quân hậu trên bàn cờ $N \times N$ sao cho không có hai quân hậu nào tấn công nhau, nghĩa là: không có hai quân hậu nào nằm cùng một hàng, cùng một cột, hoặc cùng một đường chéo.

Variables

- Mỗi quân hậu (queen) sẽ được đặt trên một hàng (row) và cột (column).
- Xét cột là index của quân hậu.

Hard constraints

- Không có 2 quân hậu cùng 1 dòng.
- Không có 2 quân hậu cùng 1 cột.
- Không có 2 quân hậu cùng 1 đường chéo.

Soft constraints

- Không có.

Test Problem 3: School Timetabling

Xếp thời khoá biểu cho trường học

Lập lịch các buổi học vào các khung giờ và phòng học để tạo ra một thời khóa biểu hợp lý hơn cho giáo viên và học sinh.

Refresh Solve Score: 0hard/10soft

By room By teacher By student group

Timeslot	Room A	Room B	Room C
Monday 08:30 - 09:30	Geography by C. Darwin 10th grade 30	Chemistry by M. Curie 9th grade 17	
Monday 09:30 - 10:30	Biology by C. Darwin 9th grade 18	French by M. Curie 10th grade 29	
Monday 10:30 - 11:30		Physics by M. Curie 10th grade 22	English by I. Jones 9th grade 20
Monday 13:30 - 14:30		Chemistry by M. Curie 10th grade 28	English by I. Jones 9th grade 21
Monday 14:30 - 15:30		Physics by M. Curie 9th grade 19	History by I. Jones 10th grade 31
Tuesday 08:30 - 09:30	Math by A. Turing 10th grade 26	Spanish by P. Cruz 9th grade 23	

Variables

- Mỗi buổi học (lesson) cần gán (timeslot, và room) hợp lệ.

Hard constraints

- Một phòng học chỉ có tối đa 1 buổi học tại một thời điểm.
- Một giáo viên chỉ dạy tối đa 1 buổi học tại cùng một thời điểm.
- Một học sinh chỉ học tối đa 1 buổi học tại cùng một thời điểm.

Soft constraints

- Giáo viên thích giảng dạy cố định tại 1 phòng học.
- Giáo viên thích dạy các buổi học liên tiếp và không thích các khoảng trống giữa các buổi học.
- Nhóm học sinh không thích các buổi học liên tiếp của cùng một môn học.

Test Problem 4: Call Center Realtime Scheduling

Lập lịch thời gian thực cho trung tâm cuộc gọi

Lập lịch các cuộc gọi đến cho tổng đài viên dựa trên kỹ năng yêu cầu, đồng thời giảm thiểu thời gian chờ của Call center realtime scheduling

Assign incoming calls to agents based on required skills and minimize the customer waiting time.

► Solve Score: 0hard/-4894soft

Agent	Incoming calls
Ann Life Insurance	609-297-3096 Life Insurance Waiting: 0m 3s In progress: 0m 0s
Beth Car Insurance	132-804-4630 Car Insurance Waiting: 0m 0s In progress: 0m 9s
Carl Property Insurance	271-147-9315 Property Insurance Waiting: 0m 17s In progress: 0m 2s 167-67-2440 Property Insurance Waiting: 0m 21s Estimated waiting: 0m 7s
Dennis Life Insurance	43-706-6081 Life Insurance Waiting: 0m 0s In progress: 0m 23s 413-985-299 Life Insurance Waiting: 0m 15s Estimated waiting: 0m 46s 721-86-4597 Life Insurance Waiting: 0m 6s Estimated waiting: 0m 96s
Elsa Car Insurance	684-457-2171 Car Insurance Waiting: 0m 7s In progress: 0m 6s
Francis Property Insurance	706-179-4000 Property Insurance Waiting: 0m 0s In progress: 0m 4s 861-715-9690 Property Insurance Waiting: 0m 2s Estimated waiting: 0m 5s 964-341-6923 Property Insurance Waiting: 0m 12s Estimated waiting: 0m 15s 353-549-1216 Property Insurance Waiting: 0m 7s Estimated waiting: 0m 25s 861-559-8551 Property Insurance Waiting: 0m 1s Estimated waiting: 0m 35s
Gus Life Insurance	

Variables

- Cuộc gọi (call) cần gán cho Tổng đài viên(agent) có đầy đủ kỹ năng (skill) yêu cầu.

Hard constraints

- Cuộc gọi phải được phân cho tổng đài viên có đủ kỹ năng yêu cầu (noRequiredSkillMissing).
→ Chỉ gán cuộc gọi cho tổng đài viên có đủ kỹ năng.

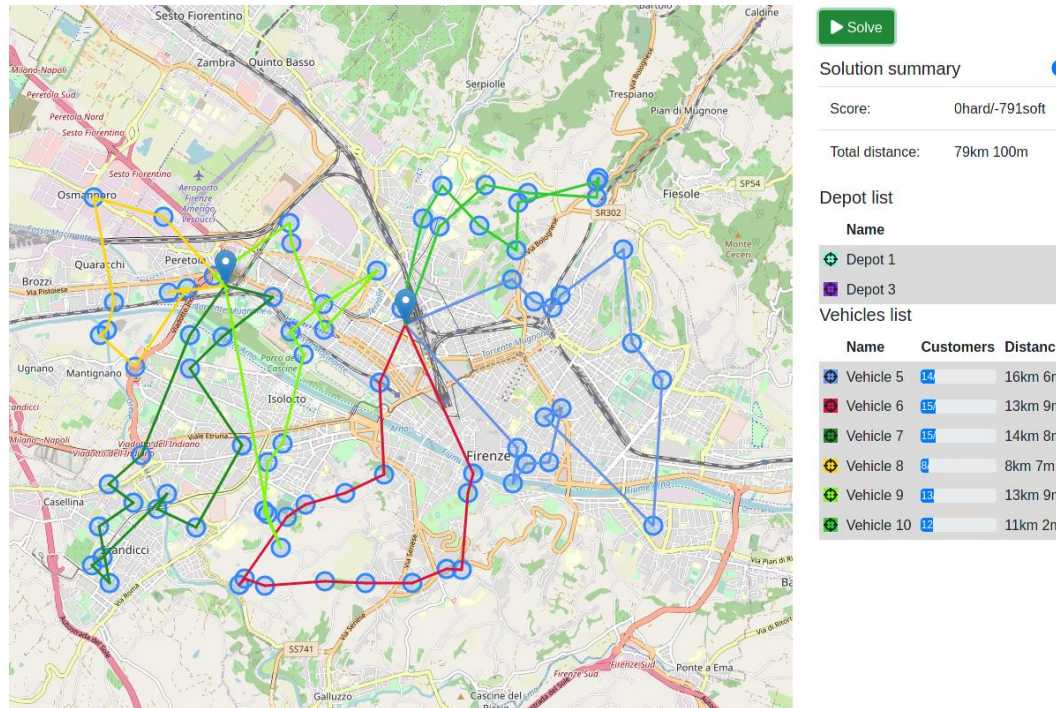
Soft constraints

- Thời gian chờ của khách hàng là ngắn nhất (minimizeWaitingTime).
→ Ưu tiên giảm các cuộc gọi đang chờ lâu.

Test Problem 5: Vehicle Routing

Điều phối lộ trình phương tiện

Tìm các tuyến đường tối ưu nhất cho một đội xe.



Variables

- Mỗi khách hàng (customer) cần được xe (vehicle) phục vụ tại một vị trí (location).

Hard constraints

- Mỗi xe không được chở quá sức chứa quy định của xe.

Soft constraints

- Quãng đường di chuyển của xe là ngắn nhất có thể.

Demo một số thực nghiệm

- Demo các bài toán thực tế với Python-Constraint và OptaPlanner.

Benchmark số trạng thái và thời gian chạy trên bài toán N-Queens

N	Số trạng thái (N!)	Số nghiệm	Brute-force	CSP (Python-constraint)	CSP (Python-constraint) 1 nghiệm	CSP (OptaPlanner) 1 nghiệm
4	24	2	0.05ms	0.28ms	0.18ms	5s
8	40,320	92	9.68ms	36.2ms	2.14ms	5s
10	3,628,800	724	259ms	718ms	1.63ms	5s
12	479,001,600	14,200	9s	14s	5.87ms	5s
14	87,178,291,200	365,596	5m37s	31.7s	7.18ms	5s
16	20,922,789,888,000		-	-	3.70ms	5s
32	$\sim 2.63 \times 10^{35}$		-	-	16.11ms	5s

Chạy trên Macbook Pro 2GHz Intel Core i5, 16GB RAM

Python 3.10

OptaPlanner 9.44

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Phương pháp giải
- ✓ Thực nghiệm
- **Kết luận & Hướng phát triển**
- Nguồn tham khảo



Kết luận & Hướng phát triển

Kết luận

- **Lập luận theo ràng buộc (Constraint Reasoning)** là một cách tiếp cận mạnh mẽ để giải quyết nhiều bài toán trong AI và khoa học máy tính như xếp lịch, trò chơi ô chữ, bài toán tối ưu hóa.
- **Mô hình CSP** (Constraint Satisfaction Problem) cho phép biểu diễn bài toán dưới dạng **biến – miền giá trị – ràng buộc**, giúp **giảm đáng kể không gian tìm kiếm** và **tiết kiệm tính toán** so với việc duyệt toàn bộ trạng thái.
- Qua các **ví dụ minh họa** (crossword puzzles, 8-queens, timetabling, ...), tiểu luận đã chứng minh rằng **kỹ thuật này vừa đơn giản, vừa có thể áp dụng linh hoạt** vào nhiều bài toán thực tế.
- Việc sử dụng **công cụ và thư viện** (như Python-constraint, OptaPlanner) hỗ trợ quá trình xây dựng, triển khai và so sánh các phương pháp giải, giúp củng cố lý thuyết bằng thực nghiệm.

Kết luận & Hướng phát triển

Hướng phát triển

- **Mở rộng bài toán:** Thử nghiệm CSP trên các **bài toán lớn, phức tạp hơn** như tối ưu hóa chuỗi cung ứng, lập lịch đa mục tiêu, hoặc bài toán thiết kế hệ thống.
- **Ứng dụng thực tế:** Phát triển **ứng dụng web hoặc phần mềm minh họa**, cho phép người dùng nhập bài toán CSP (biến, miền, ràng buộc) và quan sát lời giải được tính toán tự động.
- **Phân tích hiệu suất:** So sánh sâu hơn **hiệu quả của nhiều thuật toán giải CSP** (backtracking, consistency algorithms, local search) trên cùng một bài toán để đánh giá ưu/nhược điểm của từng cách tiếp cận.

Agenda

- ✓ Giới thiệu
- ✓ Cơ sở lý thuyết
- ✓ Phương pháp giải
- ✓ Thực nghiệm
- ✓ Kết luận & Hướng phát triển
- **Nguồn tham khảo**



Nguồn tham khảo chính

- Chapter 4: Reasoning with Constraints - Poole, David L., and Alan K. Mackworth. Artificial Intelligence: foundations of computational agents. Cambridge University Press, 2010.
(<https://www.cs.ubc.ca/~poole/aibook/3e/html/ArtInt3e.Ch4.html>)
- CSE 5692: Constraint Reasoning - Debasis Mitra. Florida Institute of Technology.
(<https://cs.fit.edu/~dmitra/Constraints/>)
- OptaPlanner Use Cases
(<https://www.optaplanner.org/learn/useCases/index.html>)

Thank You!



Knowledge Based Systems
Reasoning with Constraints

GVHD: TS. Nguyễn Hùng Sơn

HVTH: Nguyễn Ngọc Đình – 2441862007

Hà Anh Dũng – 2441862005

*Every task involves constraint,
Solve the thing without complaint;
There are magic links and chains
Forged to loose our rigid brains.
Structures, strictures, though they bind,
Strangely liberate the mind.*

– James Falen